NEWS INTELLIGENCE ENGINE

Technical Specification & Architecture Document

Version 1.0 | March 2026

A backend intelligence platform that scrapes, debiases,
and structures world events into machine-readable intelligence clusters
for B2C news delivery and downstream intelligence tools.

1. Project Overview

The News Intelligence Engine is a containerised backend platform that ingests content from news outlets, blogs, and social media; filters noise and bias; and outputs structured EventCluster objects that can power multiple downstream products.

Rather than passing raw, opinionated article text to an LLM and asking for a summary, the engine extracts structured facts upstream — stripping narrative framing before any language model is involved. The result is a fact-based, bias-mitigated data layer that is genuinely reusable across products.

1.1 Problem Statement

Existing news aggregators are firehoses: they surface volume, not clarity. Users face three compounding problems:

- Bias by omission — a story covered by 10 ideologically similar sources feels authoritative even if contested.
- Fake news — viral claims spread faster than corrections, and most platforms lack contradiction-detection.
- No context — breaking news is meaningless without historical grounding. Users cannot evaluate significance without knowing what led to an event.

1.2 Solution

The engine solves these problems at the data layer, not the UI layer:

- Bias mitigation — bias is filtered before the LLM sees the text, not post-hoc by prompting.
- Fake news detection — claims are cross-referenced against a historical vector database before being included in a cluster.
- Historical context — Qdrant retrieves semantically similar past clusters at query time, giving every event a timeline.

1.3 Downstream Products

The engine is built as a platform. Its structured output feeds:

- B2C News App — 60-word neutral summaries, contradiction highlights, confidence scores.
- Stock Prediction Model — entity sentiment timelines, anomaly flags, co-occurrence signals.
- Finance Intelligence Tool — structured event data filtered by asset or sector.
- Personal Assistant Agent — entity-filtered morning briefings via REST query.
- OSINT Monitor — real-time anomaly stream via WebSocket, plotted on a map.

2. System Architecture

The system is a containerised pipeline built around a single core abstraction: the EventCluster. Every component either produces, enriches, or consumes EventClusters.

2.1 Tech Stack

Services marked (external) are pre-existing on the host machine and are not managed by the repo's docker-compose.

Orchestration	Apache Airflow (LocalExecutor) — external service, DAG scheduling and parallel scraping
Message Broker	RabbitMQ — containerised, decouples fast scraping from slow ML inference
AI / Logic	LangGraph (Python) — multi-agent workflows
Local Inference	Ollama (Llama 3) — external service, zero-cost high-volume local LLM calls
Relational DB	PostgreSQL — containerised, canonical truth and final EventCluster payloads
Vector DB	Qdrant — containerised, semantic routing and historical context
Delivery	FastAPI — containerised, REST and WebSocket endpoints
Async PG Driver	asyncpg — async Postgres driver used by all live services
ORM	SQLAlchemy (asyncio) — async ORM layer, single schema definition
Async HTTP	httpx — async client for Ollama API calls
Async MQ	aio-pika — async RabbitMQ client for pipeline workers

2.2 Pipeline Overview

Data flows through four sequential stages, each with a clear responsibility boundary:

- Stage 1 — Ingestion & Routing: Airflow scrapes sources; Qdrant clusters by semantic similarity.
- Stage 2 — Velocity & Anomaly Detection: Lightweight NER and statistical models flag anomalies without invoking LLMs.
- Stage 3 — Multi-Agent Verification: LangGraph agents extract facts, check claims, resolve contradictions, and write the final summary.
- Stage 4 — Storage & Delivery: FastAPI exposes the enriched EventCluster to downstream consumers.

3. Repository Structure

Airflow and Ollama are external services running on the host machine. The repo's docker-compose.yml only manages: PostgreSQL, Qdrant, RabbitMQ, the pipeline worker, and the API.

3.1 Folder Layout

```
news-engine/
├── docker-compose.yml         # postgres, qdrant, rabbitmq, api, pipeline only
├── .env
├── .env.example
│
├── pipeline/                  # LangGraph workers + ingestion
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── ingestion/
│   │   ├── scrapers/
│   │   │   ├── rss.py
│   │   │   ├── financial.py
│   │   │   └── social.py
│   │   └── trafilatura_parser.py
│   ├── routing/
│   │   ├── embedder.py        # chunks + embeds incoming articles
│   │   └── cluster_router.py  # Qdrant similarity check, spawn/attach logic
│   ├── anomaly/
│   │   ├── ner_runner.py
│   │   ├── velocity_detector.py
│   │   └── source_divergence.py
│   ├── agents/
│   │   ├── graph.py           # LangGraph graph definition
│   │   ├── extractor.py       # Agent 1
│   │   ├── fact_checker.py    # Agent 2
│   │   ├── resolver.py        # Agent 3
│   │   └── judge.py           # Agent 4
│   └── workers/
│       └── consumer.py        # RabbitMQ consumer, triggers LangGraph
│
├── api/                       # FastAPI service
│   ├── Dockerfile
│   ├── requirements.txt
│   ├── main.py
│   ├── routers/
│   │   ├── clusters.py
│   │   ├── entities.py
│   │   ├── search.py
│   │   └── stream.py
│   └── dependencies.py        # shared FastAPI deps (DB session injection)
│
├── db/                        # Schema + migrations
│   ├── models.py              # SQLAlchemy ORM — single source of truth
│   ├── session.py             # Sync SessionLocal (Alembic + seed scripts only)
│   ├── async_session.py       # AsyncEngine + AsyncSessionLocal (all live code)
│   ├── migrations/            # Alembic versions
│   │   └── versions/
│   └── seed/
```

```
│        └── outlet_spectrum.json
│
├── repositories/              # All DB operations live here
│   ├── base.py                # AsyncBaseRepo — shared session context manager
│   ├── cluster_repo.py
│   ├── entity_repo.py
│   ├── fact_repo.py
│   ├── contradiction_repo.py
│   └── source_repo.py
│
├── qdrant/
│   ├── setup.py               # One-off sync collection init
│   └── qdrant_repo.py         # Wraps AsyncQdrantClient
│
└── shared/
    ├── clients.py             # Initialises all async clients at startup
    ├── ollama_client.py       # httpx.AsyncClient wrapper
    ├── rabbitmq_client.py     # aio-pika wrapper
    └── logger.py
```

3.2 Repository Pattern

Routers and agents never write raw SQL or ORM queries inline. All database access goes through the repositories/ layer. This keeps query logic in one place and makes both the agents and the API routers easy to read.

```
# In an agent
from repositories.entity_repo import EntityRepo
entity = await EntityRepo().resolve_canonical('Nvidia Corp')

# In an API router
from repositories.cluster_repo import ClusterRepo
clusters = await ClusterRepo().get_trending(limit=20)
```

3.3 Async Session Design

The codebase uses two separate session factories with strict rules about where each is used.

db/session.py	Sync SessionLocal. Used ONLY by Alembic migrations and one-off seed scripts. Never imported by running services.
db/async_session.py	AsyncEngine + AsyncSessionLocal. Used by all live code — pipeline workers, agents, API routers. Built on asyncpg.

Mixing sync and async sessions causes subtle connection pool bugs that are hard to trace. The file separation makes the rule explicit and visible.

3.4 Async Client Stack

```python
# db/async_session.py
from sqlalchemy.ext.asyncio import create_async_engine, AsyncSession,
async_sessionmaker

engine = create_async_engine(settings.DATABASE_URL)  # postgresql+asyncpg://...
AsyncSessionLocal = async_sessionmaker(engine, expire_on_commit=False)



# repositories/base.py
from contextlib import asynccontextmanager
from db.async_session import AsyncSessionLocal

class AsyncBaseRepo:
    @asynccontextmanager
    async def get_session(self):
        async with AsyncSessionLocal() as session:
            try:
                yield session
                await session.commit()
            except Exception:
                await session.rollback()
                raise



# qdrant/qdrant_repo.py
from qdrant_client import AsyncQdrantClient

class QdrantRepo:
    def __init__(self):
        self.client = AsyncQdrantClient(url=settings.QDRANT_URL)

    async def search_similar(self, embedding, top_k=5): ...
    async def get_historical_context(self, embedding, top_k=3): ...
    async def upsert_cluster(self, cluster_id, embedding, payload): ...



# shared/clients.py — initialised once at FastAPI lifespan startup
qdrant   = QdrantRepo()
rabbitmq = RabbitMQClient()   # aio-pika
ollama   = OllamaClient()     # httpx.AsyncClient
```

4. Data Model

The schema is the contract between the pipeline and every downstream product. All tables live in PostgreSQL. Vector representations live in Qdrant.

4.1 Core Tables

EventCluster

The central object. One row per real-world event.

```
EventCluster {
  id                UUID         PRIMARY KEY
  headline          TEXT         -- auto-generated neutral headline
  summary           TEXT         -- 60-word LLM output (from structured data only)
  status            ENUM         -- BAKING | PROCESSING | PUBLISHED | STALE
  anomaly_flags     TEXT[]       -- VELOCITY_HIGH | UNVERIFIED_ORIGIN | ...
  confidence_score  FLOAT        -- 0-1, based on source count + diversity
  created_at        TIMESTAMP
  updated_at        TIMESTAMP
}
```

Entity (Global, Persistent)

Entities are not owned by clusters. They persist across all clusters and are referenced via a junction table. This enables cross-cluster entity tracking and co-occurrence analysis.

```
Entity {
  id              UUID         PRIMARY KEY
  name            TEXT         -- canonical form: 'NVIDIA Corporation'
  type            ENUM         -- PERSON | ORG | LOCATION | ASSET | LEGISLATION
  aliases         TEXT[]       -- ['Nvidia Corp', 'NVDA']  (trigram-indexed)
  first_seen      TIMESTAMP
  last_seen       TIMESTAMP
  total_mentions  INT
}
```

ClusterEntityAppearance (Junction)

How a persistent Entity appears within a specific EventCluster.

```
ClusterEntityAppearance {
  id              UUID
  cluster_id      UUID         FK -> EventCluster
  entity_id       UUID         FK -> Entity
  role            TEXT         -- 'decision maker' | 'subject' | 'location'
  sentiment_min   FLOAT        -- most negative source sentiment toward this entity
  sentiment_max   FLOAT        -- most positive
  sentiment_avg   FLOAT
  mention_count   INT
}
```

ClusterFact

Atomic verified facts extracted before the LLM is invoked. These are the ground truth of the cluster.

```
ClusterFact {
  id                      UUID
  cluster_id              UUID         FK -> EventCluster
  fact_text               TEXT         -- 'Rate held at 5.25%'
  is_contested            BOOLEAN
  supporting_sources      INT
  contradicting_sources   INT
}
```

ClusterContradiction + ContradictionPosition

Contradictions are structured as a topic with N positions — not a binary A vs B. Each position records which source holds it and how many other sources agree.

```
ClusterContradiction {
  id          UUID
  cluster_id  UUID         FK -> EventCluster
  topic       TEXT         -- 'future rate trajectory'
}

ContradictionPosition {
  id                  UUID
  contradiction_id    UUID         FK -> ClusterContradiction
  position_text       TEXT         -- 'pause signals end of hikes'
  source_outlet       TEXT         -- 'WSJ'
  source_url          TEXT
  sentiment_score     FLOAT
  supporting_count    INT          -- sources sharing this position
}
```

ClusterSource

Every raw source attached to a cluster. The spectrum_score is the outlet's editorial position, not the article's sentiment.

```
ClusterSource {
  id               UUID
  cluster_id       UUID         FK -> EventCluster
  url              TEXT
  outlet_name      TEXT
  spectrum_score   FLOAT        -- -1.0 (left) to +1.0 (right), editorial
  sentiment_score  FLOAT        -- -1.0 (negative) to +1.0 (positive), article
  published_at     TIMESTAMP
  raw_text         TEXT
  is_primary       BOOLEAN
}
```

4.2 Qdrant Vector Documents

Historical context is a Qdrant-only concern. Postgres does not store history — it is fetched at query time by embedding the new cluster and retrieving semantically similar past events.

```
{
  vector:  <embedding of cluster summary>,
  payload: {
    cluster_id:    'uuid',
    headline:      'Fed holds rates at 5.25%',
    entities:      ['Federal Reserve', 'Jerome Powell'],
    date:          '2024-01-31',
    anomaly_flags: [],
    confidence:    0.87
  }
}
```

5. Data Pipeline

5.1 Stage 1 — Ingestion & Routing

Airflow DAGs continuously scrape RSS feeds, financial wires, and targeted social lists using trafilatura. Every new article is immediately chunked, embedded, and compared against Qdrant.

- Cosine similarity > 0.85 to a cluster from the last 24 hours: article is silently attached to that EventCluster.
- Cosine similarity < 0.85: a new empty EventCluster is spawned in Postgres and enters BAKING status.

The baking window allows the cluster to accumulate sources before expensive ML inference is triggered.

5.2 Stage 2 — Velocity & Anomaly Detection

While a cluster is baking, lightweight statistical models run without invoking LLMs. These act as a cheap gating mechanism.

- Entity Spikes: fast NER detects if entity mentions exceed 3 standard deviations above the 7-day moving average. Flag: ANOMALY_VELOCITY_HIGH.
- Source Divergence: if a cluster has significant social volume but zero mainstream news after 30 minutes. Flag: UNVERIFIED_ORIGIN.

High-velocity anomalies can trigger immediate webhooks to downstream consumers, bypassing the rest of the pipeline.

5.3 Stage 3 — Multi-Agent Verification (LangGraph)

Once baked, RabbitMQ pushes the raw cluster to Python workers. Four agents run sequentially.

Agent 1: The Extractor

Reads raw sources in parallel. Extracts atomic claims, financial metrics, and entity mentions into a strict JSON schema. This is the only agent that reads raw article text. All subsequent agents work from structured data only.

Entity deduplication happens here: the agent normalises entity names to canonical form, queries Postgres for an exact or fuzzy match, and resolves to an existing Entity ID or creates a new one.

```
# Entity resolution flow
canonical_name = llm_normalise('Nvidia Corp')  # -> 'NVIDIA Corporation'

# 1. Exact match
SELECT id FROM Entity WHERE name = 'NVIDIA Corporation'
```

```
# 2. Alias match
SELECT id FROM Entity WHERE aliases @> ARRAY['NVIDIA Corporation']

# 3. Fuzzy match (pg_trgm)
SELECT id FROM Entity WHERE similarity(name, 'NVIDIA Corporation') > 0.7

# 4. No match -> INSERT new Entity
```

Agent 2: The Fact-Checker

Takes extracted claims and queries the Qdrant historical database. Compares new claims semantically against established facts from past clusters. Flags claims where no historical support exists or where direct contradiction is detected. Outputs a MANIPULATION_PROBABILITY score per claim.

Agent 3: The Resolver

Deduplicates entities across sources. Builds the final contradiction map — grouping positions by topic with N positions per topic. Explicitly documents unresolved conflicts rather than silently merging them.

Agent 4: The Judge

Receives the fully structured fact object — not raw articles. Writes a 60-word neutral summary. Because the LLM is summarising structured data rather than opinion-laden text, narrative bias cannot enter the output.

```
# What the Judge receives (not raw articles)
{
  'verified_facts': ['Rate held at 5.25%', 'Vote was 11-1'],
  'entities': [{'name': 'Federal Reserve', 'role': 'decision maker'}],
  'sentiment_range': {'min': -0.6, 'max': 0.3},
  'contradictions': [{
    'topic': 'future rate trajectory',
    'positions': [
      {'text': 'pause signals end of hikes', 'outlet': 'WSJ'},
      {'text': 'door open to further increases', 'outlet': 'Bloomberg'}
    ]
  }],
  'historical_context': [<top 3 Qdrant results>]
}
```

5.4 Stage 4 — Storage & Delivery

The LangGraph worker writes the enriched EventCluster to Postgres. FastAPI exposes the data lake to downstream consumers.

6. API Endpoints

Detailed request/response schemas are deferred to a subsequent design session. The endpoints below define the surface area.

6.1 B2C News App

```
GET  /api/v1/clusters/trending
     Returns published clusters ordered by velocity score.

GET  /api/v1/clusters/:id
     Full EventCluster payload including facts, contradictions, sources.

GET  /api/v1/clusters/:id/context
     Historical EventClusters from Qdrant related to this event.
```

6.2 Intelligence Tools

```
GET  /api/v1/search?entities=NVIDIA&sentiment=negative
     Entity-filtered cluster search for personal assistant / finance tools.

GET  /api/v1/entities/:id/timeline
     All clusters an entity has appeared in, ordered by date.

GET  /api/v1/entities/:id/cooccurrence
     Entities that consistently appear alongside this one (useful for
     stock prediction co-movement signals).
```

6.3 Real-Time

```
WS   /api/v1/stream/anomalies
     WebSocket stream of ANOMALY_VELOCITY_HIGH and UNVERIFIED_ORIGIN
     clusters in real time. Used by OSINT monitor for map plotting.
```

7. MVP Build Plan

The full pipeline is built in phases. Agents 2 and 3 (Fact-Checker and Resolver) are v2 work — they should not block the first shippable version.

Week 1	Airflow DAGs scraping RSS + financial wires into Postgres raw storage.
Week 2	Qdrant integration — embedding pipeline and EventCluster routing by similarity threshold.
Week 3	FastAPI with /clusters/trending and /clusters/:id endpoints.
Week 4	Single LLM summarisation agent (Agent 4 only). Structured input schema from basic NER.
Week 5	NER anomaly detection (entity spike + source divergence flags).
Week 6	Agent 1 (Extractor) with entity deduplication and full structured fact object.
v2	Agent 2 (Fact-Checker), Agent 3 (Resolver), RabbitMQ, contradiction map, MANIPULATION_PROBABILITY.

RabbitMQ should be deferred until throughput pain is observed. Airflow's task queuing handles MVP volume without a separate broker.

8. Deferred Design Decisions

The following decisions are intentionally deferred until the MVP is validated. They are noted here so they are not forgotten.

- API contract — full request/response schemas for all endpoints.
- Extractor prompt engineering — the exact prompt and output schema for Agent 1. This is the most critical single piece of the system.
- Scoring logic — how confidence_score, spectrum_score, and MANIPULATION_PROBABILITY are calculated numerically.
- Entity graph — cross-entity relationship modelling beyond co-occurrence counts.
- Social media ingestion — rate limits, auth, and platform-specific parsing for Reddit/X.
- Outlet spectrum database — the editorial spectrum_score values for known outlets need a curated seed dataset.
- RabbitMQ introduction — deferred until throughput warrants it.